

Laboratório de Programação Orientada a Objetos I

Modelagem, sintaxe e testes

REALIZAR APÓS Módulo 2 / Semana 2

Módulos 1 a 2 | 120 minutos | 100 pontos

Curso principal: Laboratório de Programação Orientada a Objetos - Parte 1 | <https://pt.coursera.org/learn/lab-poo-parte-1>

Conteúdo verificável: OO; herança; UML; compilação/interpretação; Java/Python; boas práticas; depuração; pytest

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

Objetos e invariantes

Uma Conta tem titular e saldo inicialmente zero. Defina atributos, operações depositar e sacar, e o invariante. Escreva em Python uma implementação que impeça saque superior ao saldo e valores não positivos. Justifique por que modificar saldo diretamente prejudica o contrato.

Desenvolvimento / justificativa / esquema:

Modelagem e herança

Modele uma biblioteca com Livro, Exemplar, Pessoa e Emprestimo. Um Livro pode ter vários exemplares e cada empréstimo associa uma pessoa a um exemplar. Indique associações e multiplicidades em um diagrama de classes. Discuta se Exemplar deve herdar de Livro. Defina como representar empréstimos encerrados sem perder o histórico.

Desenvolvimento / justificativa / esquema:

Depuração e teste

O método abaixo deveria acrescentar o valor uma única vez. Localize o defeito, corrija-o e escreva dois testes pytest, incluindo um caso de limite. Explique por que um teste que só verifica que o programa não lança erro é insuficiente.

```
def acrescentar(saldo, valor):  
    if valor > 0:  
        saldo += valor  
    saldo += valor  
    return saldo
```

Desenvolvimento / justificativa / esquema:

Do modelo ao programa

Projete Pedido e ItemPedido: cada item possui descrição, quantidade positiva e preço unitário não negativo. Implemente o cálculo do total sem armazenar um total redundante. Mostre o resultado para itens (2, 7.50) e (3, 4.00). Compare o caminho entre código fonte e execução em Java e Python e identifique onde um erro de nome pode aparecer.

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - Objetos e invariantes (15 pontos)

O invariante é $\text{saldo} \geq 0$. Toda mutação deve passar por operações que validem o contrato. Uma convenção de atributo interno ajuda a sinalizar essa restrição em Python.

```
class Conta:
    def __init__(self, titular):
        self.titular = titular
        self._saldo = 0
    def depositar(self, valor):
        if valor <= 0:
            return False
        self._saldo += valor
        return True
    def sacar(self, valor):
        if valor <= 0 or valor > self._saldo:
            return False
        self._saldo -= valor
        return True
```

Atribuir um saldo negativo externamente contorna as verificações. Aceitar encapsulamento mais rigoroso em Java ou outra solução no vocabulário do curso.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - Modelagem e herança (25 pontos)

Livro 1—0..* Exemplar; Exemplar 1—0..* Empréstimo; Pessoa 1—0..* Empréstimo. Cada empréstimo tem exatamente um exemplar e uma pessoa, data inicial e data final opcional. Pode haver no máximo um empréstimo ativo por exemplar. Exemplar possui referência a Livro: é uma cópia física de uma obra, não uma subclasse com substituição comportamental. Encerrar registra a data final e mantém a associação. O diagrama deve distinguir multiplicidade histórica da regra de exclusão entre empréstimos ativos.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Depuração e teste (25 pontos)

Para valores positivos há soma dupla; negativos também são aceitos pela soma incondicional. Sob o contrato de ignorar valores não positivos:

```
def acrescentar(saldo, valor):  
    return saldo + valor if valor > 0 else saldo  
  
def test_positivo():  
    assert acrescentar(10, 4) == 14  
  
def test_zero():  
    assert acrescentar(10, 0) == 10
```

Adicionar teste negativo é desejável. Não lançar erro não demonstra que saldo final obedece ao contrato. Uma solução que rejeita entradas inválidas explicitamente também é aceitável se esse comportamento for documentado e testado.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - Do modelo ao programa (35 pontos)

```
class ItemPedido:  
    def __init__(self, descricao, quantidade, preco):  
        self.descricao = descricao  
        self.quantidade = quantidade  
        self.preco = preco  
    def subtotal(self):  
        return self.quantidade * self.preco  
  
class Pedido:  
    def __init__(self, itens):  
        self.itens = list(itens)  
    def total(self):  
        return sum(item.subtotal() for item in self.itens)
```

O contrato pressupõe entradas validadas; aceitar validação no construtor. Total=15+12=27. Recalcular evita inconsistência ao alterar um item. Java normalmente compila para bytecode executado pela JVM; Python normalmente compila internamente para bytecode executado pelo interpretador. Em Java muitos nomes inválidos são detectados na compilação; em Python a resolução pode falhar ao executar o caminho. Não confundir interpretado com ausência absoluta de compilação.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em progresso.csv. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.